

Okres (1) - (2) Code-Review,
wstępny projekt protokołu i
serializacja sieciowa

Raport poprawności

na start programu:

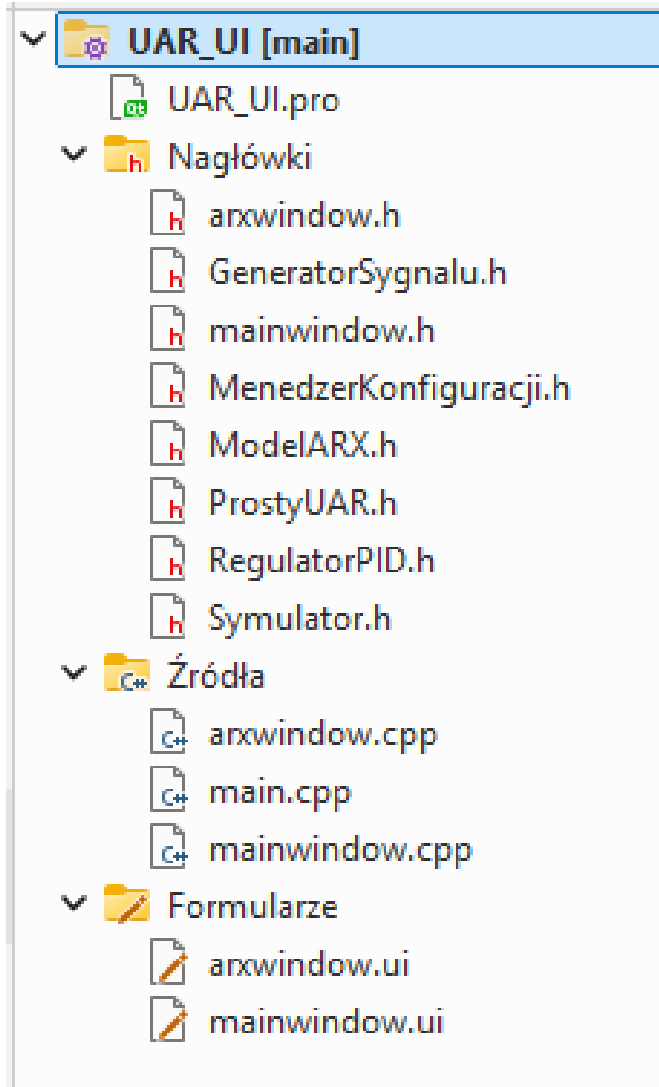
- brak wyboru sposobu całkowania, część "I" nieaktywna
- wartość regulowana ulega zmianie dopiero po wyłączeniu ograniczeń
- Okno wczytywania konfiguracji z pliku otwierało się dwukrotnie, wczytanie zmieniało tylko wartości w oknie arx
- ograniczenia na pid, czy potrzebne?
- reset części całkującej ustawia T_i na 0, czy tak ma być?

Błędy celowe

```
double deltaCzas = TT / 1000.0; // Konwersja ms na s  
//double deltaCzas = TT / 10000.0; <- tak było
```

```
//I = akum_zew * Ti / uchyb; tak było  
I = akum_zew * 1.0/Ti;
```

Dwie najgrosze



```
switch (tryb) {  
case SINUS: {  
    if (TRZ <= 0)  
        return S;  
    double czestotliwosc = 1.0 / TRZ;  
  
    return A * sin(2.0 * M_PI * czestotliwosc * czasAkumulowany) + S;  
}  
case PROSTOKAT: {  
    if (TRZ <= 0)  
        return S;  
    double okres = TRZ;  
    double pozycjaWOkresie = fmod(czasAkumulowany, okres);  
    double czasWlaczenia = okres * p;  
  
    // Dla prostokąta: 0 do A  
    if (pozycjaWOkresie < czasWlaczenia) {  
        return A + S; // stan wysoki  
    } else {  
        return S; // stan niski  
    }  
}  
default: {  
    return S;  
}  
}
```

$$w_i = \begin{cases} A + S, & \text{gdy } (i \bmod T) < pT \\ S, & \text{w przeciwnym wypadku} \end{cases}$$
$$w_i = A \sin\left(\frac{i \bmod T}{T} 2\pi\right) + S$$

Dwie najlepsze

```
Tryb tryb;  
double A;           // amplituda  
double S;           // składowa stała  
double p;           // wypełnienie (0.0 - 1.0)  
double TRZ;         // okres rzeczywisty [s]  
int TT;             // taktowanie symulacji [ms]  
double czasAkumulowany; // Akumulowany czas w sekundach  
int ostatniKrok;     // Ostatni krok dla śledzenia zmiany interwału
```

```
double symuluj(double wartoscZadana)  
{  
    double uchyb  
        = wartoscZadana  
        - wartwyjsc_poprzedni; //obliczamy różnice między zawartością zadaną a poprzednim wyjściem obiektu  
  
    double PID = pid.symuluj(uchyb); //podajemy uchyb na regulatora PID  
  
    double WartWyjsc = obiekt.symuluj(PID); //podajemy sygnał sterujący do ModeluARX  
  
    wartwyjsc_poprzedni = WartWyjsc; //zapisujemy wyjście do obliczeń uchybu  
  
    return WartWyjsc; //zwracamy wartość  
}
```

```
//Serie  
seriaP = new QLineSeries();  
seriaI = new QLineSeries();  
seriaD = new QLineSeries();  
seriaUchyb = new QLineSeries();  
seriaRegulator = new QLineSeries();  
seriaZad = new QLineSeries();  
seriaRegulowana = new QLineSeries();  
  
//Tytuły  
seriaZad->setName("Wartość zadana");  
seriaRegulowana->setName("Wartość regulowana");  
seriaP->setName("P");  
seriaI->setName("I");  
seriaD->setName("D");  
seriaUchyb->setName("Uchyb");  
seriaRegulator->setName("Sterowanie u");  
  
//Połączenie z symulatorem  
connect(&symulator, &SymulatorUAR::krokWykonany, this, &MainWindow::onKrokWykonany);  
  
//Konfiguracja Wykresu Głównego  
QChart *Mainchart = new QChart();  
QChartView *MainchartView = new QChartView(Mainchart);  
Mainchart->setAnimationOptions(QChart::NoAnimation);  
MainchartView->setMinimumSize(600, 400);  
Mainchart->addSeries(seriaZad);  
Mainchart->addSeries(seriaRegulowana);  
Mainchart->setTitle("Zadana i Regulowana");  
  
mainX = new QValueAxis();  
mainY = new QValueAxis();  
mainX->setTitleText("Czas [s]");  
mainY->setTitleText("Wartość");  
mainX->setTickCount(11);  
Mainchart->addAxis(mainX, Qt::AlignBottom);  
Mainchart->addAxis(mainY, Qt::AlignLeft);  
seriaZad->attachAxis(mainX);  
seriaZad->attachAxis(mainY);  
seriaRegulowana->attachAxis(mainX);  
seriaRegulowana->attachAxis(mainY);  
  
MainchartView->setRenderHint(QPainter::Antialiasing);  
ui->horizontalLayout_5->addWidget(MainchartView);
```